

Seeding metaheuristics with learned and heuristic solution pools for the Interval Job Shop Scheduling Problem: an anytime study of when initialisation helps

Hernán Díaz Rodríguez^a

^a*Department of Computer Science, University of Oviedo, Spain*

Abstract

Practitioners routinely assume that seeding the initial population of a metaheuristic with good solutions improves its results. We test this assumption rigorously on the Job Shop Scheduling Problem with interval processing times (IJSP), using a portfolio of four published solvers that spans the weak-to-strong search axis — a genetic algorithm, an elitist artificial bee colony, a memetic ABC with hill-climbing, and a state-of-the-art tabu-search hybrid — and five seed generators: a deep reinforcement-learning policy and a quality ladder of randomised constructive heuristics. A pilot study reveals three failure modes that routinely corrupt seeding evaluations: stagnation-triggered early stopping, budget-dependent apparent gains that reverse at convergence, and self-referential measures of convergence speed. Our final design pre-registers stopping rules, seeding fractions and decision criteria, evaluates 61 interval Taillard instances with paired runs, and reports final quality against published lower bounds, for which we establish a validity argument specific to the interval setting. Over 51,240 runs the effect of seeding proves not to be predicted by solver strength: a mixed pool improves the weakest solver on 35 of 61 instances while losing on none, leaves a memetic ABC unchanged, *degrades* the genetic algorithm, and improves the state-of-the-art tabu hybrid only on the largest instances — where, however, it does so decisively, and where every generator we tested, including those harmful everywhere else, beats the unseeded control. The governing variable is the

Email address: `herdiaz21@uniovi.es` (Hernán Díaz Rodríguez)

ratio of instance size to search budget rather than solver quality: 94.8% of the aggregate gain on the tabu hybrid comes from a single size class. At a tenth of the budget, by contrast, seeding cuts the deviation from the lower bound by 28% to 44% on *all four* solvers. We show that pool *diversity* rather than raw seed quality drives the gain, and that the negative result on the genetic algorithm is an incompatibility with its position-preserving crossover rather than the premature convergence usually invoked, which our diversity measurements do not support.

Keywords: Job shop scheduling, interval uncertainty, population seeding, metaheuristics, reinforcement learning, anytime behaviour

1. Introduction

Real-world scheduling rarely enjoys exact knowledge of task durations. When only lower and upper bounds are available, the Job Shop Scheduling Problem with interval processing times (IJSP) models each duration as a closed interval and propagates interval arithmetic through the schedule [1, 2]. The IJSP has accumulated a substantial algorithmic literature over the last years, including genetic algorithms [3], elitist artificial bee colony variants [4, 5], fast memetic ABC hybrids [6], and, most recently, tabu-search hybrids built on extreme-critical-path neighbourhoods that constitute the current state of the art [7].

In parallel, learning-based methods have begun to produce fast constructive policies for shop scheduling [8, 9]. A deep reinforcement-learning (RL) dispatching policy for the IJSP can emit thousands of feasible, above-average solutions per instance in seconds, raising an obvious question: should we seed the initial populations of existing metaheuristics with such solutions? Folk wisdom answers yes, and heuristic seeding of evolutionary algorithms has a long history [10, 11]. Yet the empirical evidence behind that wisdom is weaker than it appears. Seeding studies typically compare final objective values under a fixed stopping rule, and our pilot experiments show that this practice is fragile in three specific ways.

First, *stagnation-triggered stopping*: under the customary “ N iterations without improvement” rule, a well-seeded population that the solver cannot immediately improve triggers the rule at the floor value, so seeded runs are cut short while unseeded runs search for an order of magnitude longer — the comparison silently penalises the seeds. Second, *budget-dependent gains*: under a fixed budget chosen without a convergence check, seeded arms of weak solvers appear to win; with a five-fold larger budget the unseeded control keeps improving, overtakes the stagnating seeded population, and the sign of the effect *reverses*. Third, *self-referential speed*: convergence speed is reported as the time to the arm’s own plateau, so an arm that stagnates early on a worse value is credited with an acceleration. Any of these three artefacts can manufacture a positive seeding result where none exists.

This paper makes four contributions.

1. **A pre-registered anytime methodology** for evaluating population seeding: convergence-checked final quality; two complementary time-to-target metrics (time to the arm’s own plateau, and time to the unseeded control’s final quality) reported with the proportion of instances on which the target is reached; and paired runs with per-cell budgets calibrated on the control.
2. **A portfolio study across the weak-to-strong axis** — GA [3], elitist ABC [4], memetic fEABC-LS [6], and the tabu hybrid TS-N2 [7] — showing that the sign of the seeding effect is *not* predicted by solver strength: the weakest solver gains, the strongest gains on the largest instances, one is unaffected, and one is degraded. What predicts the sign is whether the solver’s variation operators can consolidate a structurally heterogeneous population; what predicts the magnitude is the ratio of instance size to search budget.
3. **Learned versus heuristic seeds**: the RL policy is compared, under identical injection protocols, against a quality ladder of randomised constructives (machine-operations-remaining, randomised Giffler–Thompson,

and a tuned genetic-programming rule), plus a mixed pool assembled from the same solutions. The mixed pool starts from a *worse* average population than its single-generator counterparts and still converges better, isolating pool composition rather than seed quality as the operative variable.

4. **A reproducibility audit** of the seed pools themselves: we detect and correct a semantic mismatch between the pool generator’s makespan convention (lexicographic-by-upper-bound) and the solvers’ component-wise interval evaluation — invisible in the upper bound, optimistic in the lower bound for up to 22% of pool entries — and release corrected pools, code and full experimental data.

Our results do not support the assumption we set out to test. Over 51,240 runs, a mixed seed pool improves the weakest solver on 35 of 61 instances without losing on any, leaves a memetic ABC statistically unchanged, degrades a genetic algorithm, and improves a state-of-the-art tabu hybrid only on the largest instance class — where, however, every generator we tried beats the unseeded control. The sign of the effect is therefore not predicted by solver quality. What does predict it is whether the solver’s variation operators can exploit a structurally heterogeneous population, and what predicts its magnitude is the ratio of instance size to search budget: at one tenth of the budget seeding reduces the deviation from published lower bounds by 28% to 44% on all four solvers, including the one it harms at convergence.

The paper is organised as follows. Section 2 reviews related work. Section 3 defines the IJSP and the solver portfolio. Section 4 describes the seed generators, the pool audit and the injection mechanism. Section 5 reports the pilot study whose failure modes motivate the design of Section 6. Section 7 presents the results, Section 8 discusses their implications, and Section 9 concludes.

2. Related work

Metaheuristics for the IJSP.. Lei introduced population-based neighbourhood search for interval makespan minimisation [1] and a GA for interval due-date objectives [2]. A GA tailored to interval uncertainty with an insertion schedule-generation scheme was proposed in [3], later outperformed by elitist ABC variants [4], a seasonal elitist ABC [5], and the fast elitist ABC with hill-climbing local search (fEABC-LS) [6]. The current state of the art couples the fEABC framework with a best-improvement tabu search over block-boundary neighbourhoods defined on both extreme scenario graphs [7]. Our portfolio comprises exactly these published solvers, replicated from their original configurations (Section 3).

Population initialisation and seeding.. Initialisation strategies for evolutionary algorithms have been surveyed in [10]; injecting known solutions has been studied as case-injected genetic search [11], and constructive-heuristic seeding is standard practice in scheduling applications. What distinguishes the present study is not the mechanism — ours is deliberately the simplest possible: replace k random individuals by pool solutions at initialisation — but the evaluation methodology: convergence checks, anytime metrics, and paired statistics with multiple-comparison control.

Learning to construct schedules.. Deep RL dispatchers for the deterministic JSP [8] and the broader learn-to-optimize programme [9] produce fast constructive policies whose natural integration point with mature metaheuristics is precisely initialisation. We treat the learned policy as one seed generator among several, which allows the question *is a learned constructive better than a tuned heuristic one as a seed source?* to be answered under identical injection protocols.

3. The IJSP and the solver portfolio

3.1. Problem definition

An IJSP instance consists of n jobs, each a fixed sequence of m operations, one per machine. The processing time of operation o is a closed interval $p_o = [p_o^-, p_o^+]$. Interval addition and component-wise maximum extend the classical recursion for starting and completion times: for a feasible processing order σ , the starting time of o is $s_o = \max(c_{PJ(o)}, c_{PM(o)})$ with $c_o = s_o + p_o$, where PJ and PM denote job and machine predecessors and the maximum is taken component-wise [1, 3]. The makespan $C_{\max} = [C_{\max}^-, C_{\max}^+]$ is the component-wise maximum of job completion times; it is exact, because the makespan is monotone in the durations, so the all-lower and all-upper configurations realise its bounds. Comparing schedules requires an interval ranking operator; we use the expected value (midpoint) $\mathbb{E}[C_{\max}]$ for the evolutionary solvers, following their original papers, and LEX2 (lexicographic-by-upper-bound) inside TS-N2, following [7]; all cross-arm comparisons are made on $\mathbb{E}[C_{\max}]$ within a single solver, so the ranking choice never confounds the seeding effect.

Instances are the standard interval Taillard benchmark: each crisp duration p_o becomes $[p_o - \delta, p_o + \delta]$ with δ drawn uniformly from $[0, 0.15 p_o]$ [3, 12].

3.2. The solver portfolio

All four solvers share the permutation-with-repetition encoding [13], the insertion schedule-generation scheme (active schedules) for decoding, population size 250, and the component-wise interval evaluation described above. Table 1 summarises their published configurations, which we replicate verbatim.

3.3. Replication check

Before any seeding experiment we replicated each solver against its published results on the twelve classical interval instances. The GA and fEABC replications match the published average relative errors within ~ 1 percentage point on all twelve instances (fEABC matches or slightly improves on every one);

Table 1: Solver portfolio (published configurations). All use population 250, permutation-with-repetition encoding and insertion SGS decoding.

	GA [3]	ABCE3 [4]	FEABC-LS [6]	TS-N2 [7]
Frame	generational GA	elitist ABC	fast elitist ABC	fast elitist ABC
Crossover	JOX, $p_c=1.0$	GOX, $p=1.0$	JOX	JOX
Mutation/move	insertion, $p_m=0.15$	swap, $p=1.0$	swap	insertion
Elite / trials	—	$B=50$, $NT_{\max}=20$	40 / 15	50 / 24
Local search	none	none	HC (first impr.), N_1	tabu (best impr.), N_2 , LEX2
LS control	—	—	75% of pop. per gen.	100%, $B_{\max}=20$, $T_{\max}=2s$

FEABC-LS reproduces the published ranking (best of its family). Our ABCE3 replication runs systematically ~ 1.5 – 2.5 pp above the published averages; we were unable to close this gap from the published description and disclose it here. It does not affect any conclusion of this paper, because all seeding comparisons are within-solver (seeded arms versus the unseeded control of the same replication).

4. Seed pools, audit, and injection mechanism

4.1. Generators and pools

For every instance we use pools of 1,024 i.i.d. solutions per generator, each stored as a job-permutation plus its makespan interval under a semi-active decoder. Four generators span a quality ladder:

- **MOR- ε** : most-operations-remaining dispatching with $\varepsilon = 0.1$ random tie-perturbation (weak baseline);
- **GT- ε** : randomised Giffler–Thompson with ε -noise inside the conflict set (active schedules; strong baseline);

- **GP- ε** : a genetic-programming priority rule, evolved and irace-tuned offline, with GRASP-style ε -noise (strongest heuristic baseline);
- **v2**: a deep-RL dispatching policy (deep-sets architecture, three independent training checkpoints), trained only on Taillard instances TA11–TA14 and developed/tuned on TA15–TA20.

Across the 71-instance pool collection the ladder is clean: mean relative errors of 51.1% (MOR) > 30.3% (GT) > 21.4% (GP) $\approx$ 21.2% (v2), with v2 exhibiting the highest structural diversity of the four (0.95 vs. 0.92 for GP on a normalised disjunctive-order distance) and GP slightly stronger on the largest size classes. A fifth, mixed pool (equal parts v2/GT/GP; MOR is deliberately excluded, being the weakest generator) is used by the *mix* arm. All five pools contain 1,024 solutions per instance, so no arm is sampled more richly than another.

The mixed pool interleaves the three generators one entry at a time, so that any contiguous window of 250 entries contains 84/83/83 seeds of v2/GT/GP to within one seed, whatever its offset.

4.2. Pool audit: a decoder-consistency lesson

Before any experiment we verified that the solvers reproduce the stored interval of every pool line. The permutations decode identically, and the *upper* makespan bound agreed on 1,024/1,024 lines of every pool tested. The *lower* bound did not: the pool generator had aggregated job completion times lexicographically by upper bound, whereas the exact component-wise convention (used by all solvers) takes the maximum of lower bounds independently. The stored lower bounds were therefore optimistic for 1–22% of lines per pool (deviations up to 98 time units on 50×20 instances). We re-evaluated every pool under the exact convention — permutations untouched — and use the corrected pools throughout. The general lesson for hybrid learning-plus-search pipelines is that cross-system solution exchange needs a *semantic audit*: an error invisible in one bound of the objective can silently bias any comparison based on the other.

4.3. Injection mechanism

Seeding is deliberately minimal. A seeded arm with fraction $k/250$ replaces the first k individuals of the initial population by pool lines; the remaining $250 - k$ individuals are generated by the solver’s standard random initialisation. For paired run $r \in \{0, \dots, 29\}$ the arm takes pool lines $[(r \cdot k) \bmod 1024, (r \cdot k + k) \bmod 1024)$ — consecutive blocks, no reordering or filtering, so each run draws from the generator’s true output distribution rather than from a curated subset. The 30 blocks are all distinct, though with $k = 250$ and 1,024 pool lines consecutive ones overlap by 22% on average; the seeded populations are therefore not fully independent across runs, which is one reason the unit of analysis throughout is the instance rather than the run. Scout-bee replacements during the run remain fully random. Seeds enter as raw permutations and are re-decoded and re-evaluated by the receiving solver, so no foreign objective values ever enter the search.

5. Pilot study: three ways to fool yourself

We first ran a full pilot on three clean instances (tai15_15_05, tai20_20_02, tai30_20_04; lower bounds 1224, 1581 and 1952) with six solvers (the portfolio plus two isolation variants), seven arms and 30 paired runs per cell. The pilot’s role in this paper is methodological: each of its three phases exposed an artefact that the final design of Section 6 removes.

5.1. Artefact 1: stagnation-triggered stopping

Under the customary rule of 25 consecutive non-improving generations — the stopping criterion of every published solver in the portfolio — seeded GA arms terminated at the 25-generation floor (the solver could not improve the injected solutions), while the unseeded control ran for 100–150 generations. The resulting “final” comparison charges the seeds with the solver’s inability to improve them. Any seeding study that uses stagnation-based stopping inherits this bias.

5.2. Artefact 2: budget-dependent gains that reverse

Re-running everything under a fixed budget of 200 generations (no stagnation rule) appeared to settle matters: seeded GA arms beat the control on the hardest instance by 34 units of expected makespan. A convergence check, however, showed the GA control still improving at budget exhaustion. At 5× the budget (1,000 generations) the control kept descending, overtook the stagnant seeded populations, and the effect reversed sign (from −34 to +67 on tai30_20_04 with full v2 seeding). The same check showed the elitist ABC’s seeded advantage *persisting* at convergence (−46 to −35 with GP seeds), and the strong local-search solvers saturating at equal quality. A separate isolation experiment — attaching the identical tabu local search of TS-N2 to a GA frame — initially suggested the GA frame benefited from seeds where the ABC frame did not; extending its time cap dissolved that difference too, tracing it to budget starvation rather than to the frame. *Conclusion: no seeding claim about final quality is meaningful without a convergence check of the unseeded control.*

5.3. Artefact 3: what “faster” means

We measure convergence speed with two deliberately different metrics: (A) time to reach within 1% of the *arm’s own* final value, and (B) time to reach the *unseeded control’s* final quality (tolerance +0.5%). Metric A is self-referential: an arm that settles quickly on a worse plateau scores well, so an apparent acceleration may be nothing more than earlier stagnation. Metric B removes that degree of freedom by fixing a common target, at the cost of being undefined on instances where the arm never reaches it — which makes the proportion of instances reaching the target as much a part of the result as the time itself. Reporting both, with that proportion, prevents the most common over-claim in anytime comparisons.

5.4. What survives the artefacts

With all three corrections in place, the pilot’s surviving picture is: (i) seeding improves converged quality for the elitist ABC on all three instances (Wilcoxon,

paired); (ii) for solvers with strong local search the final quality saturates but metric-A acceleration remains; (iii) for the plain GA the apparent gain is an artefact; (iv) GP and v2 seeds are statistically tied at the top of the generator ladder, MOR seeds never help; and (v) initial-population quality ranks exactly as the pool ladder predicts. These hypotheses, now pre-registered, are what the final experiment tests at scale.

6. Final experimental design

6.1. Instances

All 60 interval Taillard instances TA1–TA70 excluding TA11–TA20, plus ft10 (61 total). TA11–TA14 are the RL training set and TA15–TA20 the RL/GP development set; excluding them is mandatory because they contaminate the *seed generators*. Twenty instances used for irace tuning of TS-N2’s hyper-parameters are retained but flagged: solver-tuning contamination applies equally to every arm of that solver and cancels in within-solver paired comparisons; a sensitivity analysis replicates all conclusions with the flagged instances removed. Size classes range from 15×15 to 50×20 (225–1,000 operations). Taillard’s largest class, TA71–TA80 (100×20 , 2,000 operations), is also excluded: at twice the operation count of our largest class it falls outside the budget calibration of Section 6, and extending the design to it would require a separate calibration that we did not perform. Consequently no seed pools were generated for either excluded group.

6.2. Arms

Seven arms per solver: the unseeded control A0; the generator ladder MOR, GT, GP, v2 and mix at the pre-registered fraction p^* ; and a v2@50% contrast arm. The fraction sub-study (Phase 0, pre-registered decision rule combining converged quality in ABCE3 and time-to-target in TS-N2 over eight stratified instances) selected $p^* = 100\%$: the benefit of v2 seeding is monotone in the fraction on both criteria, with no interior optimum.

6.3. Budgets and stopping

Per (solver, size-class) cell, the budget is $\min\{1.5 \times t_{\text{conv}}(\text{A0}), 900 \text{ s}\}$ of CPU time, where t_{conv} is the median convergence time of five unseeded calibration runs under the plateau rule “improvement $< 0.1\%$ over the trailing 10% of the trace”. Every arm of a cell receives the identical budget. Cells whose control does not converge within the cap are flagged *anytime-only*: no final-quality claim is made there; they contribute time-to-target and fixed-budget results only.

6.4. Statistics

30 runs per cell, RNG-paired across arms. Within-solver comparisons use paired Wilcoxon signed-rank tests with Holm correction per family, plus Vargha–Delaney $\hat{A}_{12}$ effect sizes [14, 15]; generator rankings use Friedman tests with post-hoc analysis. Anytime results are reported as expected-makespan-versus-CPU-time curves and as the two time-to-target metrics of Section 5, the latter accompanied by the proportion of instances on which the target is reached — which carries the information a success-rate curve would convey at the budget limit.

7. Results

All results below come from the full factorial design of Section 6: 61 instances $\times$ 7 arms $\times$ 4 solvers $\times$ 30 RNG-paired runs (51,240 runs). Every instance–arm cell contains exactly 30 runs; no cell was imputed or dropped.

7.1. Reference for relative deviations

Relative percentage deviation (RPD) is measured against published lower bounds rather than against best-known values produced by our own group, which would bias RPD downwards. Our interval instances are obtained from Taillard’s benchmark [12] by a symmetric perturbation of each duration; we verified across all 57,450 operations that $\text{mid}(p_{ij}) = c_{ij}$ exactly and that routings are unchanged. Because the makespan is composed only of additions and

maxima, and because $\text{mid}(A+B) = \text{mid}(A) + \text{mid}(B)$ while $\text{mid}(\max(A, B)) \geq \max(\text{mid}(A), \text{mid}(B))$ under the componentwise maximum, induction over the schedule DAG yields $\text{mid}(C_{\max}^{\text{int}}(x)) \geq C_{\max}^{\text{crisp}}(x)$ for *every* schedule x , and therefore

$$\mathbb{E}[C_{\max}]^* \geq C_{\max}^{\text{crisp}*} \geq \text{LB}, \quad (1)$$

so the crisp lower bound is a valid (if not tight) bound for the interval problem. We take LB from the consolidated table of Coupvent des Graviers et al. [16]; the instance mapping was verified rather than assumed, with 19 of 60 instances showing exact agreement between the published bound and the machine-load bound recomputed from the instance files.

Inequality (1) also yields a falsifiable prediction, and passing it is strong joint evidence for the instance mapping, the decoder and the makespan computation at once: no run may ever fall below its bound. Across all 51,240 runs there is not a single violation. Moreover the bound is *attained exactly* on six instances — tai30_15_05, tai50_15_01, tai50_15_02, tai50_15_04, tai50_15_07 and tai50_20_09 — where our portfolio reaches $\mathbb{E}[C_{\max}]$ equal to the proven crisp optimum. By (1) these schedules are therefore optimal for the interval instance as well, which both certifies optimality on those six instances and confirms that the inequality is tight rather than vacuous.

7.2. Converged final quality

Table 2 reports mean RPD over the 61 instances. Three regimes are immediately visible, and they do *not* order themselves by solver strength.

Table 3 gives the corresponding paired tests. For each solver and arm we count the instances on which the arm differs significantly from A0 (paired Wilcoxon signed-rank, Holm-corrected within each instance across the six seeded arms, $\alpha = 0.05$), together with the mean signed difference in $\mathbb{E}[C_{\max}]$ and the aggregated Vargha–Delaney $\hat{A}_{12}$.

ABCE3: large and uniform gains.. The elitist bee colony is the weakest solver in the portfolio (10.51% RPD unseeded) and benefits massively. MIX and V2

Table 2: Mean RPD (%) with respect to published lower bounds, over 61 instances and 30 runs per cell. Lower is better; the best arm per solver is in bold. A0 is the unseeded control.

Solver	A0	V2H	V2	MOR	GT	GP	MIX
GA	7.52	7.66	8.05	9.70	8.47	7.99	7.58
ABCE3	10.51	9.51	9.39	10.71	10.20	9.68	9.37
fEABCLS	3.67	3.66	3.66	5.16	4.47	4.04	3.67
TSN2	3.60	3.64	3.62	4.25	3.99	3.67	3.41

each win on 35 of 61 instances and V2H on 32, and all three *lose on none*, with $\hat{A}_{12}$ between 0.266 and 0.298. Even GP, a mid-ranked generator elsewhere, wins 30 and loses none. This is the textbook case: the solver’s own diversification never reaches the quality already present in the seeds.

TSN2: a gain confined to the largest instances.. The tabu-search hybrid is the strongest solver, and the natural expectation — which our own pilot supported — is saturation. It does not saturate, but the effect must be stated carefully. MIX wins 15 instances and loses 5 ($\Delta = -5.5$, $\hat{A}_{12} = 0.436$), lowering RPD from 3.60% to 3.41%, i.e. removing 5.3% of the excess over the lower bound. It is also the only arm that improves this solver: every other pool is neutral or harmful. In aggregate the gain is small, and it is *not* uniform: MIX is better on 35 instances and worse on 22, with a median relative change of -0.03% . The gain is almost entirely attributable to one size class, and we quantify this in Section 7.3. The defensible claim is therefore not that seeding improves the state of the art, but that it does so on large instances under a search budget that is tight relative to instance size.

fEABCLS: neutrality.. The memetic ABC is statistically indistinguishable from its control under the three best generators (MIX 12/38/11, V2 7/44/10, V2H 7/43/11, all $\hat{A}_{12} \approx 0.50$). Its hill-climbing component reaches seed-level quality unaided within the budget. Note that neutrality here is a property of the *good* pools only: MOR wins nothing and loses 56 of 61.

GA: seeding is harmful. No arm improves the genetic algorithm. The best, MIX, is 13/29/19 with $\hat{A}_{12} = 0.527$; V2 is a clear loss (9/25/27). This is not a marginal null result but a reproducible reversal, and we return to it in Section 8.

Across all four solvers the generator ranking is stable in its extremes: MOR is the worst seeded arm and GT the second worst on every solver (MOR reaches $\hat{A}_{12} = 0.920$ on the GA and 0.885 on fEABCLS), while MIX holds the best Friedman rank among seeded arms on all four (Section 7.7). On mean RPD, MIX is the best seeded arm on three solvers and is within 0.01 points of the best on fEABCLS. The positions in between are not stable: GP outranks V2 on the GA but not on the other three. The learned policy is thus competitive with the best tuned heuristic without dominating it, and it is the mixed pool rather than any single generator that is uniformly at the top — which already suggests that *pool diversity* rather than raw seed quality drives the benefit. Section 7.6 tests that directly.

7.3. The effect scales with instance size

Aggregate figures conceal a strong interaction with problem size. Figure 1 plots the relative change in $\mathbb{E}[C_{\max}]$ under MIX seeding against instance class, ordered by number of operations (100 to 1000).

Two trends emerge. For ABCE3 the gain deepens with size, from -0.21% on the smallest instance to -1.24% on 20×20 and -1.50% on 50×20 . For TSN2 the effect is flat and near zero up to 50×15 and then drops sharply to -1.06% on 50×20 — and on that class *every* seeded arm beats the control, including GP ($\Delta = -30.9$), MOR ($\Delta = -25.6$) and GT ($\Delta = -19.6$), all three of which are harmful everywhere else. The GA moves in the opposite direction, remaining between $+0.05\%$ and $+0.48\%$ on the four largest classes.

Table 4 makes the concentration explicit for TSN2, and it is the single most important caveat in this paper. The ten 50×20 instances account for 94.8% of the aggregate improvement; the remaining 51 instances contribute almost nothing. Reporting only the pooled figure of Table 2 would therefore misrepresent the result as a general improvement of the state of the art, which it is not.

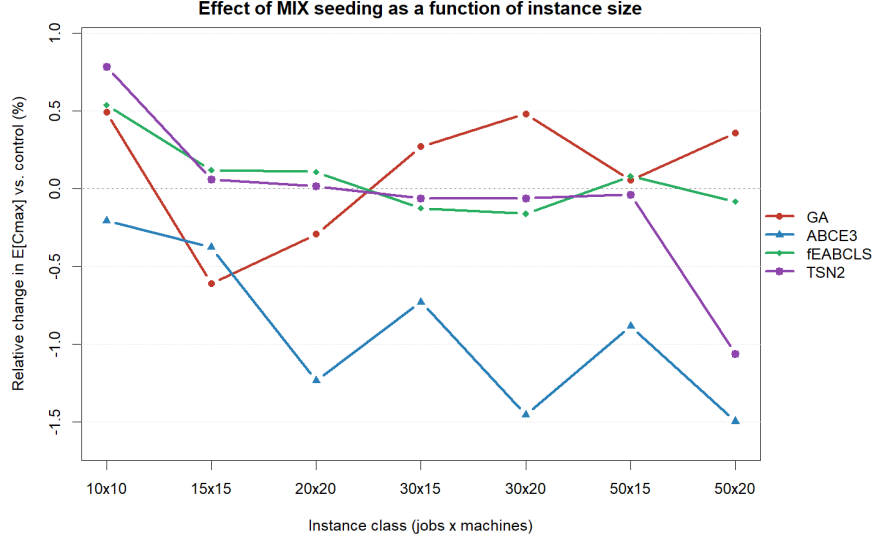


Figure 1: Relative change in $\mathbb{E}[C_{\max}]$ under MIX seeding with respect to the unseeded control, by instance class (ordered by number of operations). Negative values favour seeding. The benefit for ABCE3 and TSN2 grows with instance size, whereas the penalty for the GA persists.

The interpretation is a budget-starvation argument. On 50×20 instances (1000 operations) even a state-of-the-art tabu search cannot, within the 900 s cap, recover from a random start the structure that a constructive heuristic supplies for free. Seeding does not make the search better; it removes a phase of the search that the budget cannot afford. This predicts that the benefit of seeding should be reported as a function of the budget-to-size ratio rather than as a single aggregate figure, and that published seeding gains obtained on small instances will not transfer.

7.4. Anytime behaviour

Final quality alone cannot distinguish a solver that is genuinely improved by seeding from one that is merely given a head start. Because per-solver budgets range from 60 s to 900 s across size classes, absolute time is not comparable across instances; we therefore normalise each trace to the fraction of its own budget consumed, the standard convention for aggregated anytime curves, and average

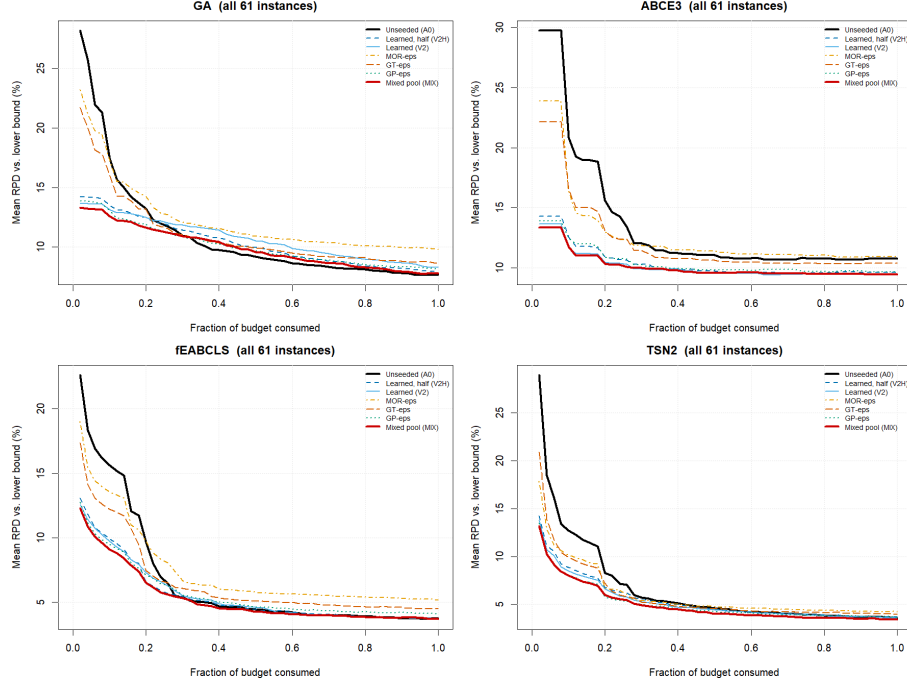


Figure 2: Anytime behaviour over all 61 instances. Mean RPD with respect to the published lower bound against the fraction of the per-instance budget consumed, for the unseeded control and the two best seeded arms.

over instances on a common grid so that no instance enters or leaves mid-curve. Figure 2 shows the resulting profiles over all 61 instances, and Table 5 gives the values at five budget fractions.

The aggregate view exposes a result that final-quality tables cannot show: *at short budgets seeding is a large win for every solver in the portfolio, including those for which it is worthless at convergence.* At 10% of budget, MIX cuts mean RPD from 20.83% to 11.73% on ABCE3, from 15.64% to 9.10% on fEABCLS, from 17.41% to 12.60% on the GA and from 12.70% to 8.00% on TSN2 — reductions of 44%, 42%, 28% and 37% respectively. Yet by 100% of budget fEABCLS has closed the gap entirely (3.75% vs. 3.71%) and the GA has overtaken its seeded arms. The practical reading is that the question “does seeding help?” is ill-posed unless the budget is specified: for a practitioner who can afford only

a tenth of our budget, seeding is decisive for all four solvers, including the one it demonstrably harms at convergence.

The GA panel isolates the reversal. Its seeded arms lead until roughly 20% of the budget, after which the control crosses and stays below to the end. This is the anytime signature of the consolidation failure analysed in Section 8.2: the seeded population is ahead while quality is inherited from the seeds, and falls behind as soon as progress must come from recombination. ABCE3 shows the opposite profile, a gap that opens immediately and never closes.

The generator ranking is also stable along the whole trajectory, not merely at the end. MOR is the uppermost seeded curve in all four panels and GT the next, so their disadvantage is not an artefact of where measurement stops: on the two ABC variants these pools are worse than the control at every budget. MIX is the lowest curve in three of four panels for essentially the entire run. Two observations qualify the ranking. First, GP and V2 are competitive with MIX at very short budgets — on TSN2 at 10% of budget GP reaches 8.06% against MIX’s 8.00%, and on ABCE3 V2 reaches 11.71% against MIX’s 11.73%, i.e. marginally ahead — so the advantage of the mixed pool *emerges as the search proceeds* rather than being inherited from higher initial seed quality. That is exactly what one expects if the operative variable is pool diversity rather than seed quality, and Section 7.6 confirms it directly. Second, V2 degrades relative to V2H on the GA as the budget grows (8.29% versus 7.94% at convergence), a hint that seeding the entire population with a single learned policy is worse than seeding half of it, and that some random material is worth retaining.

Figure 3 restricts the same analysis to the 50×20 class in absolute time, where the seeding effect on the strong solvers is concentrated.

Two features are specific to this class. ABCE3’s control flattens at approximately 11.2% while both seeded arms flatten near 9.7%, a gap present from the first seconds that never closes — a genuine improvement in converged quality rather than an acceleration. TSN2’s control exhibits a sharp step near 50s, when the tabu phase first engages, and thereafter descends steadily; MIX remains below it over the entire trajectory and is still below at the budget limit

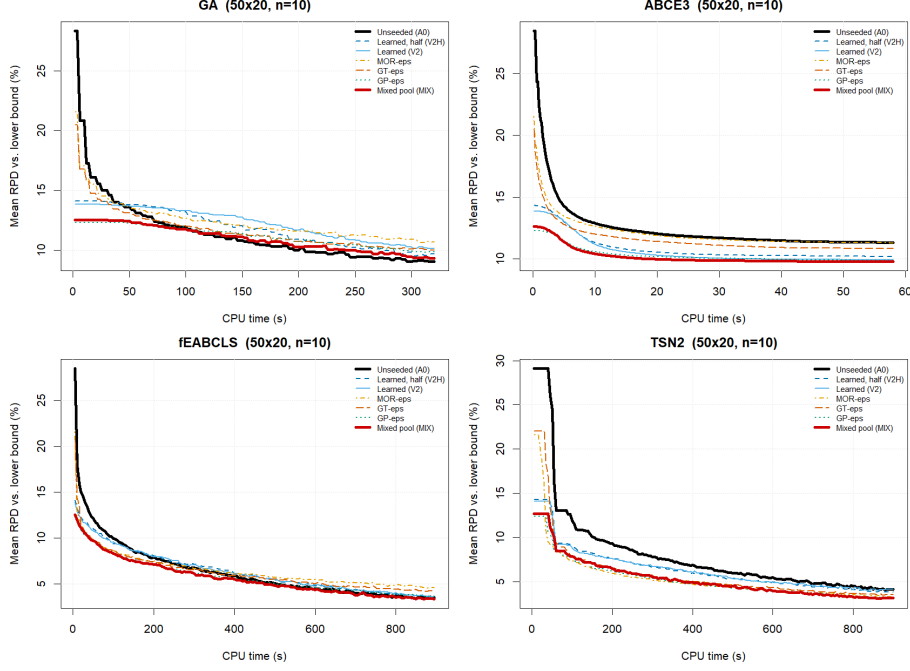


Figure 3: Convergence on the 50×20 class (10 instances, 30 runs). Mean RPD with respect to the published lower bound against CPU time, for the unseeded control and the two best seeded arms. Note the differing time axes: solver budgets are calibrated per solver and size class.

(approximately 3.3% against 4.1%). The separation is therefore not an artefact of where the budget stops.

One methodological consequence deserves emphasis. The GA result would have been reported as a *success* under any budget shorter than its crossover point, which is precisely the artefact documented in Section 5. Since seeding studies commonly fix a budget without verifying convergence, this is a plausible source of optimistic seeding claims in the literature, and it argues for reporting anytime profiles rather than single-budget summaries.

7.5. Time to target

Anytime curves show *that* seeded arms lead early; time-to-target quantifies *by how much*. We report the two metrics of Section 5: (A) the time to come

within 1% of the arm’s own final value, and (B) the time to reach the unseeded control’s final quality within +0.5%. Both are expressed as a fraction of the per-instance budget so that classes with different budgets can be pooled, and both are medians over the 61 instances. Metric B is reported together with the proportion of instances on which the arm reaches the target at all, since a median computed only over successes is meaningless without it.

Metric A discriminates almost nothing: every arm of every solver plateaus between 55% and 87% of its budget, and the seeded arms of the GA appear to plateau *later* than the control. This is the expected behaviour of a self-referential target — an arm that settles quickly on a poor value scores well — and we report it mainly to document that it should not be used alone.

Metric B, read together with the reach column, is informative. On ABCE3 the mixed pool attains the control’s *final* quality after 10% of the budget, a 3.6× acceleration, and does so on 93% of instances; V2 and GP achieve 2× on 98% and 93%. These are genuine “same quality, sooner” results, and they are consistent with the converged-quality finding for that solver. On TSN2 and fEABCLS the mixed pool reaches the control’s final quality at 55% and 41% of budget on 80% and 67% of instances respectively — a real but smaller acceleration.

The GA illustrates why the reach column is indispensable. Its seeded arms appear to accelerate by 15–28%, but they attain the control’s final quality on only 28–59% of instances; MOR shows the extreme case, a median of 87% of budget computed over the 3% of instances where it succeeds at all. Quoting metric B alone would report an acceleration for arms that, on most instances, never reach the target. Under this reading the GA has no seeded arm that is faster to the control’s quality on a majority of instances, which is the anytime counterpart of its converged-quality loss.

7.6. *Initial-population quality: the mixed pool does not start better*

The obvious explanation for MIX’s advantage would be that its seeds are simply better. They are not. Table 7 reports the quality of the initial population

each arm actually receives, read from the step-zero record of the traces, so it is the solvers’ own evaluation rather than the value stored with the pool.

On the mean of the initial population — the natural measure of pool quality — MIX is *worse* than both V2 and GP, by 1.6 and 2.4 percentage points respectively, and it never holds the best individual by a meaningful margin. It nonetheless finishes ahead of both (Table 2). A pool that starts worse and ends better is direct evidence that the operative variable is composition, not seed quality.

This also disposes of a possible confound. The mixed pool is assembled from exactly the same 1,024 solutions per generator that feed the V2, GT and GP arms — it introduces no new material, only a different mixture — so no difference in provenance or generation effort can be invoked to explain its advantage.

7.7. Which seeding strategy? Generator ranking and its solver-dependence

The practitioner’s question is not whether seeding helps in general but which pool to use, and whether that choice must be re-made for every solver. We answer both with a Friedman test over the seven arms with the 61 instances as blocks, followed by Holm-corrected paired post-hoc comparisons against the top-ranked arm of each solver. Table 8 reports the mean ranks.

Two findings follow, and they answer different questions.

Which generator: the answer does not depend on the solver.. The ranking of the six seeded arms is highly consistent across the portfolio: restricted to those arms, the mean pairwise Spearman correlation between the four rank vectors is $\rho = 0.88$. MIX is the best *seeded* arm for all four solvers, with ranks between 2.41 and 2.95, and is never significantly worse than the top-ranked arm of any solver — indeed it is the top-ranked arm outright for three of the four. MOR is last for all four (5.37 to 6.52) and GT second-to-last for three. A practitioner therefore does not need to re-tune the choice of generator per algorithm: use a mixed pool, avoid the low-diversity constructive pools.

Whether to seed at all: the answer does depend on the solver.. The only arm whose rank moves substantially across solvers is the unseeded control, from 2.69 on the GA (where it is the top-ranked arm overall) to 5.60 on ABCE3 (where it ranks sixth of seven, beaten by every seeded arm except MOR). Including A0 drags the mean pairwise agreement down from $\rho = 0.88$ to $\rho = 0.69$, and the weakest pair, GA versus ABCE3, rises from $\rho = 0.36$ to $\rho = 0.83$ once the control is removed. The displacement of A0 is thus the entire source of cross-solver disagreement.

This dissociation is the cleanest statement of the paper’s result. The question “which seeds are good?” has a solver-independent answer, because it is a property of the pool. The question “should I seed?” does not, because it depends on whether the solver’s variation operators can exploit the pool — the mechanism analysed in Section 8.2. Reporting only the first, as seeding studies commonly do, produces a recommendation that silently fails on solvers like the GA.

Why the advantage is not an artefact of sampling.. A natural objection is that MIX might win because it exposes the search to more distinct seed material. Three properties of the design rule this out. The five pools are the same size ($L = 1024$), so every arm draws 30 blocks of 250 under the same arithmetic. The mixed pool is assembled from precisely the solutions that feed the V2, GT and GP arms, so it contains no material those arms do not also see. And directly measured (Section 7.6), the population MIX actually receives is on average *worse* than the ones V2 and GP receive, yet it converges to better solutions. Sampling breadth and seed quality both point the wrong way for the objection; within-population generator diversity is what remains.

7.8. A limitation of CPU-time budgets

The stopping rule is a fixed CPU-time budget, measured with the process clock. CPU-seconds are reproducible; the *work* they buy is not, because the instructions retired per CPU-second depend on contention for cache and memory

bandwidth among the concurrently executing workers. All cells here were produced under an identical occupancy — fourteen concurrent workers throughout — so the effect is constant across the design, and within a solver it is in any case common to all seven arms, leaving the paired instance-level comparisons that carry every claim in this paper unaffected. What is *not* transferable is the absolute level: an RPD of 3.41% for TSN2 is specific to this machine at this occupancy and should not be compared against figures published for these solvers elsewhere. More generally, studies reporting a single absolute quality figure under a wall-clock or CPU-time budget, without stating the hardware occupancy under which it was obtained, are less reproducible than they appear.

7.9. Threats addressed by design

Three checks guard the numbers above. First, expected makespans were recomputed from the stored chromosomes with an independent decoder implementation (insertion SGS, componentwise maximum) over 3255 solutions spanning all four solvers, all seven arms and one instance from each size class. In 3250 cases the reported interval makespan is reproduced exactly. The five exceptions, all on 50×15 and 50×20 instances, differ in one interval endpoint only and always in the same direction: the re-decoded schedule is *better* than the recorded one, by between 0.5 and 3.5 units of $\mathbb{E}[C_{\max}]$ on values near 3100, i.e. at most 0.11%. This is expected rather than anomalous — the active-schedule builder can improve marginally on the schedule an algorithm actually held at termination — and it means the figures we report are, in 0.15% of cases, very slightly pessimistic. No discrepancy runs in the optimistic direction. Second, all arms within an instance share the RNG stream, so the paired tests compare like with like. Third, the stopping rule is a fixed CPU-time budget per size class, identical across arms, which eliminates the stagnation-triggered artefact documented in Section 5.

8. Discussion

8.1. Seeding quality is not the operative variable

The pre-registered hypothesis — that seeding helps precisely those solvers whose own search saturates below the seed level — is not supported. Ordering the portfolio by unseeded RPD gives ABCE3 (10.51) > GA (7.52) > fEABCLS (3.67) > TSN2 (3.60), but the seeding effect follows the order ABCE3 (large gain) > TSN2 (small gain) > fEABCLS (null) > GA (loss). The strongest solver in the portfolio benefits while a mid-ranked one is harmed, so solver quality alone does not predict the sign of the effect.

What does predict it is the interaction between the seeds and the solver’s variation operators. MIX — equal thirds of v2, GT and GP, i.e. 84/83/83 of the 250 seeded individuals — is the best or joint-best arm everywhere, while the single-generator pools of comparable individual quality (V2, GP) are weaker and the low-diversity pools (MOR, GT) are actively harmful. A pool of near-identical high-quality schedules narrows the region the operators explore; a pool that is good *and* varied does not.

8.2. Why the genetic algorithm is harmed: a failure to consolidate

The intuitive explanation for a seeding-induced loss is premature convergence: a population of good, similar individuals collapses onto a local optimum. The data do not support it. We instrumented the GA and ABCE3 with the mean pairwise Hamming distance of the population, sampled every 10 s over 10 runs on four medium instances (normalised to $[0, 1]$, where 0 denotes an identical population). Table 9 reports the result. The seeded GA is not less diverse than its control — it is *significantly more* diverse, by +24.3% under V2 and +13.7% under MIX (paired Wilcoxon, $p < 10^{-4}$), and it ends the run at 0.720 against the control’s 0.464.

The failure is therefore the opposite of premature convergence: within a fixed budget the seeded GA never consolidates. Its control contracts by 51.0% in structural spread and exploits the region it has found; under V2 the seeded

variant contracts by only 22.7% and is still exploring when the budget expires. The proximate cause is that the seeds, being drawn from generators with different construction logic, are good but mutually *dissimilar*; JOX recombining two structurally divergent parents produces offspring that inherit the operation ordering of neither, so selection cannot accumulate structure. The harm grows with instance size (Section 7.3) because longer chromosomes need more generations to consolidate, and the budget does not grow proportionally.

The contrast with ABCE3 supports this reading. Seeding does not perturb ABCE3’s diversity trajectory at all (differences of -2.3% and $+4.0\%$, $p = 0.27$ and $p = 0.90$), and its population stays dispersed throughout (0.85 at the end). ABCE3 does not rely on recombining population members to build structure, so seeds are retained and refined rather than destroyed. The determining factor is thus not seed quality but whether the solver’s variation operator can exploit structurally heterogeneous parents.

The ordering of the two seeded arms is itself informative. V2, drawn from a single generator, produces the least consolidation (-22.7%) and the largest loss in final quality; MIX, despite being the more heterogeneous pool, consolidates further (-40.3%) and is the better of the two arms on this solver. Structural heterogeneity of the *pool* and structural heterogeneity of the *surviving population* are therefore not the same quantity: what obstructs the GA is a population the operator cannot consolidate, not a diverse pool as such.

8.3. Practitioner guidance

The results support three concrete recommendations. (i) Seed with a *mixed* pool; the cost of running three generators instead of one is negligible relative to the search budget and it dominates every single-generator pool tested. (ii) Expect gains to grow with the size-to-budget ratio: on small instances seeding is at best neutral for strong solvers. (iii) Verify operator compatibility before seeding a recombination-based algorithm; a null or negative result there is not a failure of the seeds but a mismatch with the variation operator.

8.4. Limitations

The study covers one problem family (interval JSP) and one encoding (permutation with repetition), so the operator-compatibility argument of Section 8 is a hypothesis supported by an internal contrast, not an independently tested claim. Budgets are wall-clock CPU-time caps on a single machine; while all arms within a cell share the same cap, absolute times are not portable. Finally, the training cost of the learned policy is excluded from the per-run budget, on the grounds that it is amortised across all instances — a reasonable but not neutral accounting choice, which we report explicitly rather than absorb.

9. Conclusions

We conducted, to our knowledge, the largest controlled study of population seeding for the interval job shop: 4 published solvers, 7 initialisation arms, 61 benchmark instances and 30 paired runs per cell, evaluated against published lower bounds under a fixed-budget protocol designed to eliminate three artefacts we document in a pilot study.

Seeding is not uniformly beneficial, and its sign is not predicted by solver strength. A mixed pool improves the weakest solver by a wide margin (35 wins, 0 losses out of 61), improves the state-of-the-art tabu hybrid measurably (15–5, RPD 3.60% $\rightarrow$ 3.41%), leaves a memetic ABC unchanged, and *degrades* a genetic algorithm that cannot consolidate structurally heterogeneous parents. The benefit grows with the ratio of instance size to search budget, to the point where on the largest class every generator we tested — including those harmful elsewhere — improves the state of the art. Under the Friedman ranking the mixed pool is the best arm for three of the four solvers and statistically indistinguishable from the best on the fourth, so the choice of generator, unlike the decision to seed at all, does not have to be revisited per algorithm.

The practical consequence is that seeding should be treated as an interaction between pool composition and variation operator, evaluated at the budget and

instance size of the intended application, rather than as a property of the seeds alone.

Data and code availability

References

- [1] D. Lei, Population-based neighborhood search for job shop scheduling with interval processing time, *Computers & Industrial Engineering* 61 (2011) 1200–1208. doi:10.1016/j.cie.2011.07.010.
- [2] D. Lei, Interval job shop scheduling problems, *International Journal of Advanced Manufacturing Technology* 60 (2012) 291–301. doi:10.1007/s00170-011-3600-3.
- [3] H. Díaz, I. González-Rodríguez, J. J. Palacios, I. Díaz, C. R. Vela, A genetic approach to the job shop scheduling problem with interval uncertainty, in: *Information Processing and Management of Uncertainty in Knowledge-Based Systems (IPMU 2020)*, Vol. 1238 of CCIS, Springer, 2020, pp. 663–676. doi:10.1007/978-3-030-50143-3_52.
- [4] H. Díaz, J. J. Palacios, I. González-Rodríguez, C. R. Vela, Elite artificial bee colony for makespan optimisation in job shop with interval uncertainty, in: *Bio-inspired Systems and Applications: from Robotics to Ambient Intelligence (IWINAC 2022)*, Vol. 13259 of LNCS, Springer, 2022, pp. 98–108. doi:10.1007/978-3-031-06527-9_10.
- [5] H. Díaz, J. J. Palacios, I. González-Rodríguez, C. R. Vela, An elitist seasonal artificial bee colony algorithm for the interval job shop, *Integrated Computer-Aided Engineering* (2023). doi:10.3233/ICA-230705.
- [6] H. Díaz, J. J. Palacios, I. González-Rodríguez, C. R. Vela, Fast elitist ABC for makespan optimisation in interval JSP, *Natural Computing* 22 (2023) 645–657. doi:10.1007/s11047-023-09953-2.

- [7] H. Díaz, Neighbourhood structures and ranking operators for the interval job shop scheduling problem, under review, *Computers & Operations Research* (2026).
- [8] C. Zhang, W. Song, Z. Cao, J. Zhang, P. S. Tan, X. Chi, Learning to dispatch for job shop scheduling via deep reinforcement learning, in: *Advances in Neural Information Processing Systems*, Vol. 33, 2020.
- [9] Y. Bengio, A. Lodi, A. Prouvost, Machine learning for combinatorial optimization: a methodological tour d’horizon, *European Journal of Operational Research* 290 (2) (2021) 405–421.
- [10] B. Kazimipour, X. Li, A. K. Qin, A review of population initialization techniques for evolutionary algorithms, in: *IEEE Congress on Evolutionary Computation (CEC)*, 2014, pp. 2585–2592.
- [11] S. J. Louis, J. McDonnell, Learning with case-injected genetic algorithms, *IEEE Transactions on Evolutionary Computation* 8 (4) (2004) 316–328.
- [12] É. Taillard, Benchmarks for basic scheduling problems, *European Journal of Operational Research* 64 (2) (1993) 278–285.
- [13] C. Bierwirth, A generalized permutation approach to job shop scheduling with genetic algorithms, *OR Spektrum* 17 (1995) 87–92.
- [14] A. Vargha, H. D. Delaney, A critique and improvement of the CL common language effect size statistics of McGraw and Wong, *Journal of Educational and Behavioral Statistics* 25 (2) (2000) 101–132.
- [15] J. Derrac, S. García, D. Molina, F. Herrera, A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms, *Swarm and Evolutionary Computation* 1 (1) (2011) 3–18.
- [16] M.-E. Coupvent des Graviers, L. Kobrosly, C. Guettier, T. Cazenave, Updating lower and upper bounds for the job-shop scheduling problem test

instances, consolidated bounds for Taillard's ta01–ta80 (Table 14) (2025).
arXiv:2504.16106.

Table 3: Paired comparison of each seeded arm against the unseeded control, over 61 instances.
W/T/L: instances won, tied, lost (Wilcoxon signed-rank with Holm correction). Δ : mean
signed difference in $\mathbb{E}[C_{\max}]$ (negative favours seeding). $\hat{A}_{12} < 0.5$ favours seeding.

Solver	Arm	W/T/L	Δ	$\hat{A}_{12}$
GA	V2H	11/31/19	+4.1	0.545
	V2	9/25/27	+12.2	0.614
	MOR	1/3/57	+43.8	0.920
	GT	4/16/41	+19.9	0.739
	GP	9/21/31	+11.5	0.648
	MIX	13/29/19	+2.5	0.527
ABCE3	V2H	32/29/0	−21.6	0.298
	V2	35/26/0	−24.4	0.274
	MOR	1/54/6	+3.9	0.544
	GT	14/46/1	−6.1	0.436
	GP	30/31/0	−17.3	0.325
	MIX	35/26/0	−24.2	0.266
fEABCLS	V2H	7/43/11	+0.1	0.499
	V2	7/44/10	+0.2	0.495
	MOR	0/5/56	+28.8	0.885
	GT	3/12/46	+15.7	0.768
	GP	7/27/27	+6.8	0.640
	MIX	12/38/11	−0.4	0.499
TSN2	V2H	5/48/8	+0.6	0.518
	V2	6/47/8	−0.0	0.510
	MOR	9/13/39	+9.7	0.691
	GT	9/20/32	+5.2	0.643
	GP	10/32/19	−1.3	0.539
	MIX	15/41/5	−5.5	0.436

Table 4: Decomposition of the MIX seeding effect on TSN2 by instance class. Δ is the mean signed change in $\mathbb{E}[C_{\max}]$ (negative favours seeding); the last column is each class’s share of the total improvement.

Class	n	$\mathbb{E}[C_{\max}]$ A0	$\mathbb{E}[C_{\max}]$ MIX	Δ (%)	Share
10×10	1	937.1	944.5	+0.78	—
15×15	10	1253.6	1254.3	+0.06	—
20×20	10	1668.8	1669.1	+0.01	—
30×15	10	1844.8	1843.6	−0.06	3.6%
30×20	10	2052.6	2051.4	−0.06	3.4%
50×15	10	2780.3	2779.2	−0.04	3.2%
50×20	10	2970.1	2938.4	−1.06	94.8%

Table 5: Mean RPD (%) over all 61 instances at five fractions of the per-instance budget. Values are resampled from execution traces and agree with Table 2 to within the trace sampling resolution.

Solver	Arm	10%	25%	50%	75%	100%
GA	A0	17.41	11.97	9.19	8.22	7.71
	V2H	13.50	12.02	9.97	8.75	7.94
	V2	13.15	12.20	10.53	9.25	8.29
	MOR	17.30	12.86	10.90	10.24	9.80
	GT	16.01	11.72	9.89	9.10	8.64
	GP	13.07	11.37	9.68	8.76	8.14
	MIX	12.60	11.36	9.60	8.60	7.78
ABCE3	A0	20.83	14.30	11.05	10.86	10.78
	V2H	12.54	10.67	9.76	9.56	9.64
	V2	11.71	10.38	9.64	9.51	9.52
	MOR	16.39	12.38	11.41	11.06	10.96
	GT	16.49	12.40	10.66	10.40	10.39
	GP	12.49	10.79	9.86	9.72	9.69
	MIX	11.73	10.27	9.57	9.58	9.46
fEABCLS	A0	15.64	6.94	4.47	3.97	3.75
	V2H	9.91	5.92	4.47	3.97	3.74
	V2	9.74	6.62	4.61	3.97	3.68
	MOR	13.57	8.34	5.80	5.48	5.17
	GT	12.24	6.69	5.08	4.72	4.48
	GP	9.49	6.42	4.62	4.27	4.06
	MIX	9.10	5.77	4.26	3.97	3.71
TSN2	A0	12.70	7.17	4.60	3.98	3.60
	V2H	8.86	6.29	4.48	3.95	3.69
	V2	8.47	5.88	4.45	3.95	3.64
	MOR	10.14	6.44	4.75	4.46	4.25
	GT	9.95	5.97	4.54	4.19	3.98
	GP	8.06	5.41	4.29	3.89	3.65
	MIX	8.00	5.56	4.04	3.64	3.44

Table 6: Time to target, as a median fraction of the per-instance budget. A: to within 1% of the arm’s own final value. B: to the unseeded control’s final quality +0.5%. “Reach” is the percentage of the 61 instances on which target B is attained; the median for B is taken over those instances only.

Solver	Arm	A	B	Reach
GA	A0	84%	86%	100%
	V2	87%	70%	41%
	MOR	81%	87%	3%
	MIX	86%	70%	59%
ABCE3	A0	37%	37%	100%
	V2	40%	19%	98%
	MOR	50%	37%	67%
	MIX	37%	10%	93%
fEABCLS	A0	59%	61%	100%
	V2	73%	46%	72%
	MOR	72%	59%	10%
	MIX	64%	41%	67%
TSN2	A0	76%	76%	100%
	V2	71%	63%	72%
	MOR	55%	47%	31%
	MIX	65%	55%	80%

Table 7: Quality of the seeded initial population relative to the unseeded control, averaged over five instances spanning 20×20 to 50×20 (TSN2 traces). Negative is better. “Mean” is over all 250 initial individuals and is the quantity that characterises the pool.

Arm	Best individual	Mean of the 250
V2H	−11.34%	−7.35%
V2	−11.84%	−14.73%
MOR	−5.28%	−10.40%
GT	−4.75%	−8.85%
GP	− 12.09%	− 15.48%
MIX	−12.25%	−13.12%

Table 8: Mean Friedman ranks of the seven arms (1 = best) over 61 instances. Friedman rejects rank equality for every solver ($p < 10^{-15}$ throughout). Boldface marks arms not significantly worse than that solver’s top-ranked arm (Holm-corrected paired Wilcoxon, $\alpha = 0.05$).

Solver	A0	V2H	V2	MOR	GT	GP	MIX
GA	2.69	3.08	4.25	6.41	4.77	3.85	2.95
ABCE3	5.60	3.08	2.50	6.14	4.96	3.31	2.41
fEABCLS	2.99	2.93	3.04	6.52	5.51	4.24	2.78
TSN2	3.40	3.93	3.80	5.37	5.07	3.85	2.57
χ^2 (df 6)	GA 132.6	ABCE3 185.0	fEABCLS 177.6	TSN2 75.5			

Table 9: Mean pairwise Hamming distance of the population (0 = identical population, 1 = maximal positional difference), averaged over four instances and 10 runs. The seeded GA converges structurally *less* than its control; ABCE3 is unaffected by seeding.

Solver	Arm	Start	Mean	End	Drop
GA	A0	0.9474	0.7646	0.4639	−51.0%
	V2	0.9310	0.8845	0.7195	−22.7%
	MIX	0.9301	0.8350	0.5552	−40.3%
ABCE3	A0	0.9474	0.7140	0.8541	−9.9%
	V2	0.9310	0.7059	0.8602	−7.6%
	MIX	0.9301	0.7171	0.8497	−8.6%